

Deploying Cowrie Honeypot with Prometheus Monitoring and Slack Alerting

(Cowrie - Prometheus - Node-exporter - cAdvisor -
Alertmanager - Slack - Docker)

Prepared by: Omar Hassan

Table of Contents

1.0 Cowrie Installation & Setup

- 1.1 Dependencies & isolated user account

- 1.2 Virtual environment & configuration

2.0 Credential & Authentication Setup

- 2.1 userdb.txt rules & realistic passwords

3.0 Network & Port Configuration

- 3.1 iptables redirection to port 22

4.0 Fake Filesystem & Realism

- 4.1 honeyfs – users, SSH keys, .env files, bash history

- 4.2 Hostname, banner, MAC address & system info spoofing

5.0 Docker & Monitoring Stack

- 5.1 Cowrie container with Prometheus output

- 5.2 node-exporter & cAdvisor deployment

6.0 Prometheus & Alertmanager

- 6.1 Scrape targets & metrics validation

- 6.2 Alerting rules – brute force, commands & container down

- 6.3 Slack webhook & end-to-end alert testing

What to expect :

This lab documentation details the deployment and advanced configuration of a Cowrie honeypot designed to log brute-force attacks and malicious shell interactions. It outlines the transition from a basic installation to a highly realistic deception environment by customizing the virtual filesystem, adjusting user credentials, modifying the hostname, and spoofing the MAC address to evade anti-virtualization techniques. Additionally, the lab covers the containerization of the monitoring stack using Docker, successfully integrating Cowrie with Prometheus, Node Exporter, and cAdvisor to continuously track both system and container-level metrics. Finally, it demonstrates the configuration of Alertmanager to trigger and send real-time notifications directly to a Slack channel whenever specific malicious activities, such as brute-force attempts or unauthorized command executions, are detected within the honeypot.

Setting up & Configuring Cowrie

"Cowrie is a medium to high interaction SSH and Telnet honeypot designed to log brute force attacks and the shell interaction performed by the attacker. In medium interaction mode (shell) it emulates a UNIX system in Python, in high interaction mode (proxy) it functions as an SSH and telnet proxy to observe attacker behavior to another system. In LLM mode, it uses large language models to generate dynamic responses to attacker commands."

The installation and setup of cowrie is pretty simple and easy to use but I will be showing more advanced features and techniques to make this honeypot seem more realistic.

1)Installing dependencies & Packages

```
$ sudo apt-get install git python3-pip python3-venv libssl-dev libffi-dev build-essential libpython3-dev python3-minimal authbind
```

You will be asked to download some packages over 200MB just press 'Y'.

2)Creating a user account

```
$ sudo adduser --disabled-password cowrie
```

This is an important step but the documentation lists its recommended so it's probably optional but when setting it up in a real environment it's an important factor.

That's because if the honeypot machine got compromised the adversary won't have any privileges with a root account.

```
$ sudo su - cowrie
```

3)Cloning the Github repository of cowrie

```
$ git clone http://github.com/cowrie/cowrie
$ cd cowrie
```

4)Setting up the virtual environment

```
$pwd
```

```
$ python3 -m venv cowrie-env
```

Make sure after pwd the working directory is /home/cowrie/cowrie

```
$ source cowrie-env/bin/activate
```

```
(cowrie-env) $ python -m pip install --upgrade pip
```

```
(cowrie-env) $ python -m pip install -e .
```

We can enable telnet but I will not be doing that right now and just focusing on the other stuff.

Let's start cowrie but note that I will just run it to showcase it working there are still a lot of configurations I will do.

```
$ source cowrie-env/bin/activate
```

```
(cowrie-env) $ cowrie start
```

P.S: The following warning may occur, this can be safely ignored, and is not the reason your Cowrie installation is not working.

```
(cowrie-env) cowrie@honeypot-VMware-Virtual-Platform:~/cowrie$ cowrie start
Join the Cowrie community at: https://www.cowrie.org/slack/

Starting cowrie: [twistd --umask=0022 --pidfile /home/cowrie/cowrie/var/run/cowrie.pid --logger cowrie.python.logfile.logger cowrie].
/home/cowrie/cowrie/cowrie-env/lib/python3.12/site-packages/twisted/conch/ssh/transport.py:110: CryptographyDeprecationWarning: TripleDES.ciphers.algorithms.TripleDES and will be removed from cryptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
  b"3des-cbc": (algorithms.TripleDES, 24, modes.CBC),
/home/cowrie/cowrie/cowrie-env/lib/python3.12/site-packages/twisted/conch/ssh/transport.py:117: CryptographyDeprecationWarning: TripleDES.ciphers.algorithms.TripleDES and will be removed from cryptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
  b"3des-ctr": (algorithms.TripleDES, 24, modes.CTR),
(cowrie-env) cowrie@honeypot-VMware-Virtual-Platform:~/cowrie$
```

Using : `$ cowrie status`

We can check if it's actually running or an error has occurred before testing.

```
  b"3des-ctr": (algorithms.TripleDES, 24, modes.CTR),
(cowrie-env) cowrie@honeypot-VMware-Virtual-Platform:~/cowrie$ cowrie status
cowrie is running (PID: 5009).
(cowrie-env) cowrie@honeypot-VMware-Virtual-Platform:~/cowrie$ S
```

Now let's have a live feed of the logs.

```
$ tail -f /var/log/cowrie/cowrie.log
```

```
(cowrie-env) cowrie@honeypot-VMware-Virtual-Platform:~/cowrie$ tail -f var/log/cowrie/cowrie.log
2026-04-17T18:25:25.499638Z [-] Python Version 3.12.3 (main, Mar 3 2026, 12:15:18) [GCC 13.3.0]
2026-04-17T18:25:25.499670Z [-] Twisted Version 25.5.0
2026-04-17T18:25:25.499680Z [-] Cowrie Version 2.9.17
2026-04-17T18:25:25.499687Z [-] Sensor UUID: 0a17a012-3a7a-11f1-bc39-000c297cc79a
2026-04-17T18:25:25.502257Z [-] Loaded output engine: jsonlog
2026-04-17T18:25:25.503974Z [twisted.scripts._twistd_unix.UnixAppLogger#info] twistd 25.5.0 (/home/cowrie/cowrie/cowrie-env/bin/python 3.12.3) starting up.
2026-04-17T18:25:25.504064Z [twisted.scripts._twistd_unix.UnixAppLogger#info] reactor class: twisted.internet.epollreactor.EPollReactor.
2026-04-17T18:25:25.510017Z [-] CowrieSSHFactory starting on 2222
2026-04-17T18:25:25.510824Z [cowrie.ssh.factory.CowrieSSHFactory#info] Starting factory <cowrie.ssh.factory.CowrieSSHFactory object at 0x7c6b91c25190>
2026-04-17T18:25:25.617389Z [-] Ready to accept SSH connections
```

Now let's test this by opening a new terminal.

```
$ ssh -p 2222 root@localhost
```

If you are using another machine to connect via SSH just replace the IP address to the IP of the Honeypot.

Notice that cowrie uses port 2222 for SSH and port 2223 for Telnet by default but we will change those to simulate a realistic environment.

After being asked for a password to connect and writing a password like "root" we can go back to our terminal where we had live logs.

```
2026-04-17T18:49:29.567399Z [cowrie.ssh.factory.CowrieSSHFactory] New connection: 127.0.0.1:58736 (127.0.0.1:2222) [session: e3fcb3799485]
2026-04-17T18:49:29.568448Z [HoneyPotSSHTransport,1,127.0.0.1] Remote SSH version: SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.15
2026-04-17T18:49:29.569964Z [HoneyPotSSHTransport,1,127.0.0.1] SSH client hassh fingerprint: aae6b9604f6f3356543709a376d7f657
2026-04-17T18:49:29.570856Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug] kex alg=b'curve25519-sha256' key alg=b'ssh-ed25519'
2026-04-17T18:49:29.570914Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug] outgoing: b'aes128-ctr' b'hmac-sha2-256' b'none'
2026-04-17T18:49:29.571073Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug] incoming: b'aes128-ctr' b'hmac-sha2-256' b'none'
2026-04-17T18:49:31.153690Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug] NEW KEYS
2026-04-17T18:49:31.154512Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug] starting service b'ssh-userauth'
2026-04-17T18:49:31.155506Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] b'root' trying auth b'none'
2026-04-17T18:49:35.417280Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] b'root' trying auth b'password'
2026-04-17T18:49:35.417886Z [HoneyPotSSHTransport,1,127.0.0.1] Could not read etc/userdb.txt, default database activated
2026-04-17T18:49:35.418296Z [HoneyPotSSHTransport,1,127.0.0.1] login attempt [b'root'/b'root'] failed
2026-04-17T18:49:36.419679Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] b'root' failed auth b'password'
2026-04-17T18:49:36.419866Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] unauthorized login: ()
```

But what if we used an empty password? It would authenticate successfully and that's because the `userdb.txt` (userdb.example if the text file wasn't found) file processes rules top-to-bottom and often includes a default wildcard rule designed to accept any credential for testing purposes.

So using this logic we can checkout `/etc/userdb.example` file using nano.

```
# Field #1 contains the username
# Field #2 is currently unused
# Field #3 contains the password
# '*' for any username or password
# '!' at the start of a password will not grant this password access
# '/' can be used to write a regular expression
#
root:x:!root
root:x:!123456
root:x:!/honeypot/i
root:x:*
tomcat:x:*
oracle:x:*
*:x:somepassword
*:x:*
```

The logic here states that any password other than root would work, let's make the password harder but at the same time easy to guess/brute force.

Most attackers use passwords lists like [rockyou.txt](#) so if you're going to choose a password make sure it's in this list and from the top 50 values at least because sometimes attackers don't go over the whole list as it's time wasting and a lot of computing power.

```
#
# Field #1 contains the username
# Field #2 is currently unused
# Field #3 contains the password
# '*' for any username or password
# '!' at the start of a password will not grant this password access
# '/' can be used to write a regular expression
#
root:x:password123
root:x:!root
root:x:!123456
root:x:!/honeypot/i
root:x:*
```

This is still probably an easy password and it should be a little harder for the attacker not to suspect anything because if it's caught on from the start that it's a honeypot the attacker would stop his attack so he doesn't showcase his plan.

```
$ cd cowrie/etc/
```

```
$ cp userdb.example userdb.txt
```

Now if you try any password other than "password123" it would deny the attacker.

Listening on port 22, right now we are listening on port 2222 and that isn't the default port for SSH although sometimes attacker would suspect that an organization would be using a default port and would suspect it's a honeypot so it's a war of psychology.

The SSH daemon runs on port 22 by default. Cowrie runs on port 2222 by default. To receive most traffic, Cowrie will need to listen on port 22. This requires two changes: First, If you have an existing SSHD on port 22 it will need to be moved to another port. Second, Cowrie will need to listen to requests on port 22.

```
$ sudo iptables -t nat -A PREROUTING -p tcp --dport 22 -j REDIRECT --to-port 2222
```

Note that you should test this rule only from another host; it doesn't apply to loopback connections.

I will be using a Kali Machine to try this test.

```
(kali㉿kali)-[~]  
$ ssh -pssh -p 22 root@192.168.25.135  
root@192.168.25.135's password:
```

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@svr04:~#

Make sure to get the honeypot's IP address using `$ ip a` and then use it
connect from the other machine.

Making the honeypot seem more realistic by changing the hostname

By default the hostname is svr04 we can change our hostname related to
something related to our organization or we can leave it as it is.

To change hostname we duplicate `cowrie.cfg.dist` to `cowrie.cfg` with the
right extension.

```
$ cp etc/cowrie.cfg.dist etc/cowrie.cfg
```

```
$ nano etc/cowrie.cfg
```

```
# Hostname for the honeypot. Displayed by the shell prompt of the virtual  
# environment  
#  
# (default: svr04)  
hostname = svr04
```

Now let's try and create a fake file architecture that would waste the
attacker's time and techniques on this honeypot.

```
root@svr-web01:~# cat /etc/passwd  
root:x:0:0:root:/root:/bin/bash  
daemon:x:1:1:daemon:/usr/sbin:/bin/sh  
bin:x:2:2:bin:/bin:/bin/sh  
sys:x:3:3:sys:/dev:/bin/sh  
sync:x:4:65534:sync:/bin:/bin/sync  
games:x:5:60:games:/usr/games:/bin/sh  
man:x:6:12:man:/var/cache/man:/bin/sh  
lp:x:7:7:lp:/var/spool/lpd:/bin/sh  
mail:x:8:8:mail:/var/mail:/bin/sh
```

We can see it's not really realistic with those accounts but we could change this stuff easily.

"Part of Cowrie is an emulated file system. Each honeypot visitor will get their own personal copy of this file system and this will be deleted when they log off. They can delete or change any file, nothing will be preserved."

The file system implementation consists of two parts: the pickle file, which mostly holds metadata for the files (filename, directory, permissions, owner, size, file type, etc) but has contents for a few files. Most files have no content.

The honeyfs directory holds user created file contents, this overrides content from the pickle file and is a quick way to have custom content."

Use this command to edit the passwd file.

```
$ nano honeyfs/cowrie/passwd
```

```
GNU nano 7.2 honeyfs/etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
sshd:x:101:65534::/var/run/sshd:/usr/sbin/nologin
phil:x:1000:1000:Phil California,,,:/home/phil:/bin/bash
```

I'm going to add some realistic cases for this demo.

```
s.mansour:x:1001:1001:Sherif Mansour - DevOps Engineer:/home/s.mansour:/bin/bash
m.zaki:x:1002:1002:Mohamed Zaki - Junior Dev:/home/m.zaki:/bin/bash
svc_backup:x:1003:1003:System Backup Service:/var/backups/scripts:/usr/sbin/nologin
db_admin:x:1004:1004:Database Administrator:/home/db_admin:/bin/bash
```

Now we have to map these account to the etc/shadow so there is consistency between the two files.

```
$ nano honeyfs/cowrie/shadow
```

```
s.mansour:$2a$10$DmqtpwliVBuhvFlUyf0fEupN501m7QzkQrUwLbHH.aTDMmfBVUche:15800:0:99999:7:::
m.zaki:$2a$10$3HHoJYH0v.HxvpnhfegK8ed80q.HddL.Lc8AjtFNEG0cBG/LaoIXC:15800:0:99999:7:::
svc_backup:$2a$10$3Ks.iNvdh36FjNPo4cUXP.wT3EZV9o27aW9dYX1dx6UCKklknpSEa:15800:0:99999:7:::
db_admin:$2a$10$vdDCBiXaPzQ/.SNsxDhGwOQRm5JdYJdo68iYmDEUpA2F4Sedc.8U6:15800:0:99999:7:::
```

The `bash_history` file shouldn't be empty and should at least have some commands to mimic the user's activity.

First I will create directories for my users then run an AI generated python script to speed up the process of creating sub directories.

```
$ mkdir -p honeyfs/home/s.mansour/.ssh
$ mkdir -p honeyfs/home/m.zaki/projects/webapp
$ mkdir -p honeyfs/home/db_admin/backups
$ mkdir -p honeyfs/root/.ssh
$ touch honeyfs/home/s.mansour/.ssh/id_rsa
$ nano honeyfs/home/s.mansour/.ssh/id_rsa
```

And adding a fake RSA Private key

```
GNU nano 7.2 honeyfs/home/s.mansour/.ssh/id_rsa *
-----BEGIN RSA PRIVATE KEY-----
MIICWwIBAAKBgElzNWwNfLrglnm489DqX8U/jd0zdJd7DnejwMda4/lrwVyhgxQe
y7y8c/vFxU8PoSl/jwJmc1Uc2Q5204xyEfr03YlH0k1W89cGCPdGjWPRltECc2Dv
sXSUV1gfW0949s+haTkBtP6PPH7aZU/ofdZmzbtyWjAMuEUXSZjtpMwZAgMBAAEC
gYAt4l/wMjJUWCsIvYbq8w+DHJHQN6JYV1WFXfH3R81jKnKwGGHXdumV7TCTj1gU
oNIkJ4DCYhgYip36UDcV3Mjj5995loU1rPs7DW0H/7ec1QsG7szZkT8aQADS2NA7
FCmbmbx5N5+7501gXgAfoaVCuCLsBgEcaWjto9Fykno1JQJBAI0R+xdHqM6N0LO
azswJQH4iTuapKqM5IHgMGsOZXHeVLZKbK01oJngUjbn/JNVFCFhCbFq9XMM7Tuo
jTXQkV8CQQCF5iLiQ3RG4ahvueWwAwTRSE2bBEc0ho7NWG3lN2YLL+puNj854Y5j
oz0U8MzyXsznpxfLIS9ESkKmUPmVy72HAKEAjEGvWu1UNsedY08F8EAK223Gy7qs
Y8gjczl6Ckay98ashN10LERd2//a3eBpWfbB3/oB0CLezLeyepk3ZKTskwJAPISb
<f8m4Df8/h1tiCkwt8VPVkh63VcySdpXjV1A7aKHLhLBMDAocaBGUzsAO/CITvg
tb3vdBYzhMgnSjgjaQJABIfbQQTVb+BiM5tdqXOLmNJ2iJGZD4pgV00Jd+Mo5ARZ
PQ4x/sBYNgGYxnfe5Zz/bidOMcaa64nz7VTv0fe0g==
-----END RSA PRIVATE KEY-----
```

We can expand to a lot of scenarios like this : `$ echo "DB_PASS=P@ssw0rd123" > honeyfs/home/db_admin/.env` which would make the `.env` file and important artifact to the attacker.

```
cowrie@honeypot-VMware-Virtual-Platform: ~/cowrie
GNU nano 7.2 honeyfs/home/db_admin/backups/.env *
db_user="DB-Admin-99"
db_pass="ADMINISTRATOR@DATABASE-092Y82T7"
OPENAI_API="3URU9D43E90D30U2EN93J-2BY2BUI3RBQWNO3XB-IVY2EYVWSB3DIVXIWIN"
Github_Auth="NSNWIDNIWU1NI0843475HBZZ"
```

I will just keep it simple as this is a lab.

Now to running the python script which will add more realism to these folders.

This script is going to be a mixture of admin, devops and developer commands to simulate the users but I also wanted to ensure it had typos as humans make mistakes , you can create your own and customize it as you prefer.

Let's now create users (not just fake names)so that attacker can try and mitigate into them.

Using the userdb.txt in the etc folder we will create the same users in the passwd file.

```
#
root:x:password123
root:x:!root
root:x:!123456
root:x:!/honeypot/i
root:x:!*
s.mansour:x:mansour4321
db_admin:x:P@ssw0rd123
m.zaki:x:123321
svc_backup:x:0123456789
```

I made them easy except s.mansour which could be a little difficult because it's not in the rockyou.txt list but there are many tools available now that allows attackers to create a personalized list for each account using information about these accounts and OSINT techniques.

P.S: Make sure to comment/remove the last line *:x:* as it accepts any username with any password.

```
(kali@kali)~[~]
$ ssh -p 22 db_admin@192.168.25.135
db_admin@192.168.25.135's password:
Permission denied, please try again.
db_admin@192.168.25.135's password:
Permission denied, please try again.
db_admin@192.168.25.135's password:

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
db_admin@svr-web01:~$
```

We can confirm now it works.

To show the contents of the file, it needs both a meta data entry (pickle) and a honeyfs file.

So to sync our file structure we will have to do this: **(MAKE SURE TO BE IN HONEYFS DIRECORY WHEN WRITING THE SECOND COMMAND)**

```
$ mkdir data
```

```
$ createfs -d 1024 -o data/fs.pickle honeyfs
```

```
$ cowrie restart
```

But after logging in as the attacker I couldn't seem to find the folders or files I created so I had to do an extra step that made it work.

```
$ fsctl data/fs.pickle
```

Then start adding your directories and files and save by writing `exit`.

If it still doesn't appear after restarting then remove the `fs.pickle` file and add it again to update it using these commands:

```
$ rm ~/cowrie/data/fs.pickle
```

```
$ cd honeyfs
```

```
$ createfs -o ../data/fs.pickle .
```

```
$ cowrie restart
```

I added two files (`.env` & `info.txt`) let's check them out now.

```
db_admin@svr-web01:~$ ls
backup
db_admin@svr-web01:~$ cd backup/
db_admin@svr-web01:~/backup$ ls
info.txt
db_admin@svr-web01:~/backup$ cat info.txt
P@ssw0rd123

db_admin@svr-web01:~/backup$ ls -la
drwxrwxr-x 1 s.mansour 1001 4096 2026-04-30 08:06 .
drwxrwxr-x 1 s.mansour 1001 4096 2026-04-30 07:49 ..
-rw-rw-r-- 1 s.mansour 1001 177 2026-04-30 08:04 .env
-rw-rw-r-- 1 s.mansour 1001 13 2026-04-30 08:06 info.txt
db_admin@svr-web01:~/backup$ timed out waiting for input: auto-logout
```

As you can see we found both our files inside (Obviously you can customize it more and have more bogus data).

But I realized something was off when I used the `-a` argument and it was that `s.mansour` is the owner of the file and that was incorrect and could give it away to the attacker that it's a honeypot.

Now let's get the UID from the `honeyfs/etc/passwd` file and use it to change the ownership of the folder.

```
s.mansour:x:1001:1001:Sherif Mansour - DevOps Engineer:/home/s.mansour:/bin/bash
m.zaki:x:1002:1002:Mohamed Zaki - Junior Dev:/home/m.zaki:/bin/bash
svc_backup:x:1003:1003:System Backup Service:/var/backups/scripts:/usr/sbin/nologin
db_admin:x:1004:1004:Database Administrator:/home/db_admin:/bin/bash
```

```
$ sudo chown -R 1004:1004 /home/cowrie/cowrie/honeyfs/home/db_admin/
```

Then to apply changes lets rm the `fs.pickle` file and add it again:

```
$ rm ~/cowrie/data/fs.pickle
```

```
$ cd honeyfs
$ createfs -o ../data/fs.pickle .
$ cowrie restart
```

```
db_admin@svr-web01:~$ ls -la backup/
drwxrwxr-x 1 db_admin 1004 4096 2026-04-30 08:06 .
drwxrwxr-x 1 db_admin 1004 4096 2026-04-30 07:49 ..
-rw-rw-r-- 1 db_admin 1004 177 2026-04-30 08:04 .env
-rw-rw-r-- 1 db_admin 1004 13 2026-04-30 08:06 info.txt
```

We can also create a SSH Banner and MOTD (Message of the day) when the attacker logs in.

```
$ nano honeyfs/etc/issue
```

```
GNU nano 7.2                                     etc/issue *
*****
*                                     *
*          NOTICE TO USERS          *
*                                     *
* This system is for the use of authorized users only. Individuals using *
* this computer system without authority, or in excess of their          *
* authority, are subject to having all of their activities on this system *
* monitored and recorded by system personnel.                            *
*                                     *
* In the course of monitoring individuals improperly using this system,   *
* or in the course of system maintenance, the activities of authorized   *
* users may also be monitored.                                            *
*****
```

```
$ nano honeyfs/etc/motd
```

```
GNU nano 7.2                                     etc/motd
ATTENTION: THIS SYSTEM IS MONITORED.

Thank you for logging in. Please remember that all actions are recorded
for security purposes. Report any suspicious behavior to the administrator.
```

```
ATTENTION: THIS SYSTEM IS MONITORED.
```

```
Thank you for logging in. Please remember that all actions are recorded
for security purposes. Report any suspicious behavior to the administrator.
db_admin@svr-web01:~$
```

Now let's change the MAC address but why is that? Malwares use anti virtualization techniques to detect that it's in a virtualized environment and change its behavior.

```
$ sudo apt update && sudo apt install macchanger -y
```

```
$ ip a
```

Get the interface name and set its state to down.

```
$ sudo ip link set ens33 down
```

```
$ sudo macchanger -m 00:03:47:A1:B2:C3 ens33
```

```
$ sudo ip link set ens33 up
```

```
honeypot@honeypot-VMware-Virtual-Platform:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 1000
    link/ether 00:0c:29:7c:c7:9a brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.25.135/24 brd 192.168.25.255 scope global dynamic noprefixroute ens33
        valid_lft 1184sec preferred_lft 1184sec
honeypot@honeypot-VMware-Virtual-Platform:~$ sudo ip link set ens33 down
honeypot@honeypot-VMware-Virtual-Platform:~$ sudo macchanger -m 00:03:47:A1:B2:C3 ens33
Current MAC: 00:0c:29:7c:c7:9a (VMware, Inc.)
Permanent MAC: 00:0c:29:7c:c7:9a (VMware, Inc.)
New MAC: 00:03:47:a1:b2:c3 (Intel Corporation)
honeypot@honeypot-VMware-Virtual-Platform:~$ sudo ip link set ens33 up
```

P.S: We got a new IP from the DHCP

- **Intel:** 00:03:47
- **Dell:** 00:14:22
- **HP:** 00:08:02

We can also change the version string in the cowie.cfg file.

```
#
# (default: "SSH-2.0-OpenSSH_6.0p1 Debian-4+deb7u2")
version = SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.4
```

Editing the CPU and memory Info, in the honeyfs folder we can find etc/meminfo and etc/cpuinfo and we can adjust the info as we want.

Sending output to Prometheus

Prometheus is 100% open source and community-driven.

It integrates with Kubernetes and other cloud and container managers to continuously discover and monitor your services. It is the second project to graduate from the CNCF after Kubernetes.

Monitor your applications, systems, and services with the leading open source monitoring solution. Instrument, collect, store, and query your metrics for alerting, dashboarding, and other use cases.

I will use docker for containerization in this lab.

Go to another terminal and **kill the cowrie instance we created (\$ sudo pkill -9 -f cowrie):**

```
$ sudo snap install docker
```

Creating a network:

```
$ sudo docker network create cowrie-net
```

Creating a volume for storing data:

```
$ docker volume create prometheus-data
```

Copying the example config

```
$ sudo mkdir -p /etc/prometheus
```

```
$ sudo cp /home/cowrie/cowrie/docs/prometheus/prometheus.yaml  
/etc/prometheus/prometheus.yaml
```

Launching Prometheus

```
$ sudo docker run -p 9090:9090 -v /prometheus-  
data/prometheus.yaml:/etc/prometheus/prometheus.yaml prom/Prometheus
```

```
honeypot@honeypot-VMware-Virtual-Platform:~$ sudo docker run -p 9090:9090 -v /prometheus-data/prometheus.yaml:/etc/prometheus/prometheus.yaml prom/promet  
heus  
time=2026-04-30T15:13:31.767Z level=INFO source=main.go:1678 msg="updated GOGC" old=100 new=75  
time=2026-04-30T15:13:31.767Z level=INFO source=main.go:744 msg="Leaving GOMAXPROCS=2: CPU quota undefined" component=automaxprocs  
time=2026-04-30T15:13:31.767Z level=INFO source=memlimit.go:198 msg="GOMEMLIMIT is updated" component=automemlimit package=github.com/KimMachineGun/autom  
emlimit/memlimit GOMEMLIMIT=3650203238 previous=9223372036854775807  
time=2026-04-30T15:13:31.767Z level=INFO source=main.go:851 msg="Starting Prometheus Server" mode=server version="(version=3.11.3, branch=HEAD, revision=  
eb173f5256d4022afba1e9bc3d19740a76859fae)"  
time=2026-04-30T15:13:31.767Z level=INFO source=main.go:856 msg="operational information" build_context="(go=go1.26.2, platform=linux/amd64, user=root@83  
aad33dd38e, date=20260427-14:45:32, tags=netgo,builtinassets)" host_details="(Linux 6.17.0-22-generic #22~24.04.1-Ubuntu SMP PREEMPT_DYNAMIC Thu Mar 26 1  
5:25:54 UTC 2 x86_64 e6a3b19be0db (none))" fd_limits="(soft=524287, hard=524288)" vm_limits="(soft=unlimited, hard=unlimited)"  
time=2026-04-30T15:13:31.934Z level=INFO source=web.go:710 msg="Start listening for connections" component=web address=0.0.0.0:9090  
time=2026-04-30T15:13:31.938Z level=INFO source=main.go:1410 msg="Starting TSDB"
```

If something fails I suggest troubleshooting with online forums like stackoverflow and avoid AI.

Now let's run cowrie with Prometheus enabled using another terminal.

```
$ docker run --name cowrie \
--network cowrie-net \
-p 2222:2222 \
-p 9000:9000 \
-e COWRIE_OUTPUT_PROMETHEUS_ENABLED=yes \
cowrie/cowrie:latest
```

```
honeypot@honeypot-VMware-Virtual-Platform:~$ sudo docker run --name cowrie --network cowrie-net -p 2222:2222 -p 9000:9000 -e COWRIE_OUTPUT_PROMET
HEUS_ENABLED=yes cowrie/cowrie:latest
/cowrie/cowrie-env/lib/python3.13/site-packages/twisted/conch/ssh/transport.py:110: CryptographyDeprecationWarning: TripleDES has been moved to cryptogra
phy.hazmat.decrepit.ciphers.algorithms.TripleDES and will be removed from cryptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
  b"3des-cbc": (algorithms.TripleDES, 24, modes.CBC),
/cowrie/cowrie-env/lib/python3.13/site-packages/twisted/conch/ssh/transport.py:117: CryptographyDeprecationWarning: TripleDES has been moved to cryptogra
phy.hazmat.decrepit.ciphers.algorithms.TripleDES and will be removed from cryptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
  b"3des-ctr": (algorithms.TripleDES, 24, modes.CTR),
2026-04-30T15:42:36+0000 [-] Reading configuration from ['/cowrie/cowrie-git/etc/cowrie.cfg.dist']
2026-04-30T15:42:36+0000 [-] Python Version 3.13.5 (main, Jun 25 2025, 18:55:22) [GCC 14.2.0]
2026-04-30T15:42:36+0000 [-] Twisted Version 25.5.0
2026-04-30T15:42:36+0000 [-] Cowrie Version 2.9.17.dev1+gcdc0770d3d
2026-04-30T15:42:36+0000 [-] Sensor UUID: bafc61de-32ec-11f1-a37e-46de337b5ff9
2026-04-30T15:42:36+0000 [-] Loaded output engine: jsonlog
2026-04-30T15:42:36+0000 [-] Loaded output engine: prometheus
2026-04-30T15:42:36+0000 [twisted.scripts._twistd_unix.UnixAppLogger#info] twistd 25.5.0 (/cowrie/cowrie-env/bin/python3 3.13.5) starting up.
2026-04-30T15:42:36+0000 [twisted.scripts._twistd_unix.UnixAppLogger#info] reactor class: twisted.internet.epollreactor.EPollReactor.
2026-04-30T15:42:36+0000 [-] CowrieSSHFactory starting on 2222
2026-04-30T15:42:36+0000 [cowrie.ssh.factory.CowrieSSHFactory#info] Starting factory <cowrie.ssh.factory.CowrieSSHFactory object at 0x701a8d7fbb60>
2026-04-30T15:42:36+0000 [-] Ready to accept SSH connections
```

I will explain what the node-exporter and cAdvisor do later on.

Running node-exporter (host metrics)

```
$ docker run -d --name node-exporter \
--network cowrie-net \
--pid host \
-v /:/host:ro \
-p 9100:9100 \
quay.io/prometheus/node-exporter:latest \
--path.rootfs /host
```

Running cAdvisor (container metrics)

```
$ sudo docker run -d --name cadvisor --network cowrie-net --
privileged -v /:/rootfs:ro -v /var/run:/var/run:rw -v /sys:/sys:ro
-v /var/snap/docker/common/var-lib-docker:/var/lib/docker:ro -p 8080:8080
gcr.io/cadvisor/cadvisor:latest
```

You may face some errors depending how you installed docker,I installed it via snap.

Running cowrie with Prometheus locally

```
$ sudo nano /home/cowrie/cowrie/etc/cowrie.cfg
```

Now use CTRL + q to search for [output_prometheus]

```
[output_prometheus]
```

```
enabled = false
```

```
port = 9000
```

```
debug = false
```

Change the value of enabled to true.

Now what exactly does node_exporter and cAdvisor add to this lab?

Node_Exporter monitors the virtual machine as a whole (it doesn't see docker or cowrie it only see the virtual environment) like CPU usage, memory metrics, network traffic and disk I/O.

cAdvisor "Container Advisor" is used differentiate between cowrie's usage and Prometheus itself.

It helps us know per container resource usage, container network throughput and restarts & health.

Both of them are not mandatory of terms of a honeypot but they will definitely be of use to us.

A problem I faced, you might realize that we successfully configured everything but when trying to SSH into the honeypot again it fails, that's because we are running cowrie right now from a container and it this docker image uses `/cowrie/cowrie-git/etc` as it's config directory not `/cowrie/cowrie/etc` so I had to copy all my configs from the previous python environment for docker to read from.

What I did is this :

```
$ sudo tee /var/snap/docker/common/var-lib-docker/volumes/6d928f784ff49b0330420e51a782eea57c478bae5c4efca2625279b3ed453ff5/_data/userdb.txt << 'EOF'
```

```
root:x:password123
```

```
root:x:!root
```

```
root:x:!123456
```

```
root:x:!/honeypot/i
```

```
root:x:!*
```

```
s.mansour:x:mansour4321
```

```
db_admin:x:P@ssw0rd123
```

```
m.zaki:x:123321
```

```
svc_backup:x:0123456789
```

```
tomcat:x:*
oracle:x:*
*:x:somepassword
EOF
```

Where I got my volume from this command : `$ sudo docker inspect cowrie`

```
{
  "Mounts": [
    {
      "Type": "volume",
      "Name": "6d928f784ff49b0330420e51a782eea57c478bae5c4efca2625279b3ed453ff5",
      "Source": "/var/snap/docker/common/var-lib-docker/volumes/6d928f784ff49b0330420e51a782eea57c478bae5c4efca2625279b3e
```

But this isn't the ideal solution.

For finding the best solution I spent over 2 hours looking for where the problem is and how to solve it and I came up with this:

```
$ sudo docker rm -f cowrie
$ sudo docker run -d \
  --name cowrie \
  --network cowrie-net \
  -p 22:2222 \
  -p 9000:9000 \
  -v /home/cowrie/cowrie/etc:/cowrie/cowrie-git/etc \
  -v /home/cowrie/cowrie/honeyfs:/cowrie/cowrie-git/honeyfs \
  -v /home/cowrie/cowrie/data:/cowrie/cowrie-git/data \
  -e COWRIE_OUTPUT_PROMETHEUS_ENABLED=yes \
  cowrie/cowrie:latest
```

Now when cowrie tries to open pickle.fs file the docker engine points to the right directory where we had our configurations.

P.S: Paths may differ from my command.

Now let's check <http://127.0.0.1/targets>

The screenshot shows the Prometheus Targets page at `http://127.0.0.1:9090/targets`. It displays two scrape pools: 'cowrie' and 'scrapers'. Each pool has a table of targets with columns for Endpoint, Labels, Last scrape, and State.

Scrape Pool	Endpoint	Labels	Last scrape	State
cowrie	http://cowrie:9000/metrics	instances="cowrie:9000" job="cowrie"	4.842s ago 2ms	UP
scrapers	http://cadvisor:8080/metrics	instances="cadvisor:8080" job="scrapers"	1.445s ago 76ms	UP
	http://node-exporter:9100/metrics	instances="node-exporter:9100" job="scrapers"	4.469s ago 13ms	UP

Now let's try logging in from our kali machine.

After logging in we will try to see the login attempt using CLI first.

```
$ curl http://localhost:9000/metrics | grep cowrie
```

```
: HELP cowrie_source_ip_created Sessions per source IP
: TYPE cowrie_source_ip_created gauge
cowrie_source_ip_created{asn_country="UNK",ip="192.168.25.133"} 1.777744788699577e+09
: HELP cowrie_source_ip_cardinality Unique source IPs seen
: TYPE cowrie_source_ip_cardinality gauge
: HELP cowrie_login_attempts_total Login attempts
: TYPE cowrie_login_attempts_total counter
cowrie_login_attempts_total{result="success",username="db_admin"} 1.0
: HELP cowrie_login_attempts_created Login attempts
: TYPE cowrie_login_attempts_created gauge
cowrie_login_attempts_created{result="success",username="db_admin"} 1.7777447936472905e+09
: HELP cowrie_password_length Password length histogram
```

We can see information about the attacker like IP and the username he tried to login as and even the result.

And now using the Prometheus GUI we can also see the same information while writing the pql(Prometheus query language).

The screenshot shows the Prometheus Query Editor. The query entered is `>_ cowrie_login_attempts_total`. Below the query, there are tabs for 'Table', 'Graph', and 'Explain'. The 'Table' tab is selected, showing a table with one row of data. Below the table, there is a section for 'Evaluation time' and a button to '+ Add query'.

Query	Table	Graph	Explain
>_ cowrie_login_attempts_total	cowrie_login_attempts_total{instance="cowrie:9000", job="cowrie", result="success", username="db_admin"}		

You can also explore the metrics of cowrie,node-exporter and cadvisor by pressing the three dots.

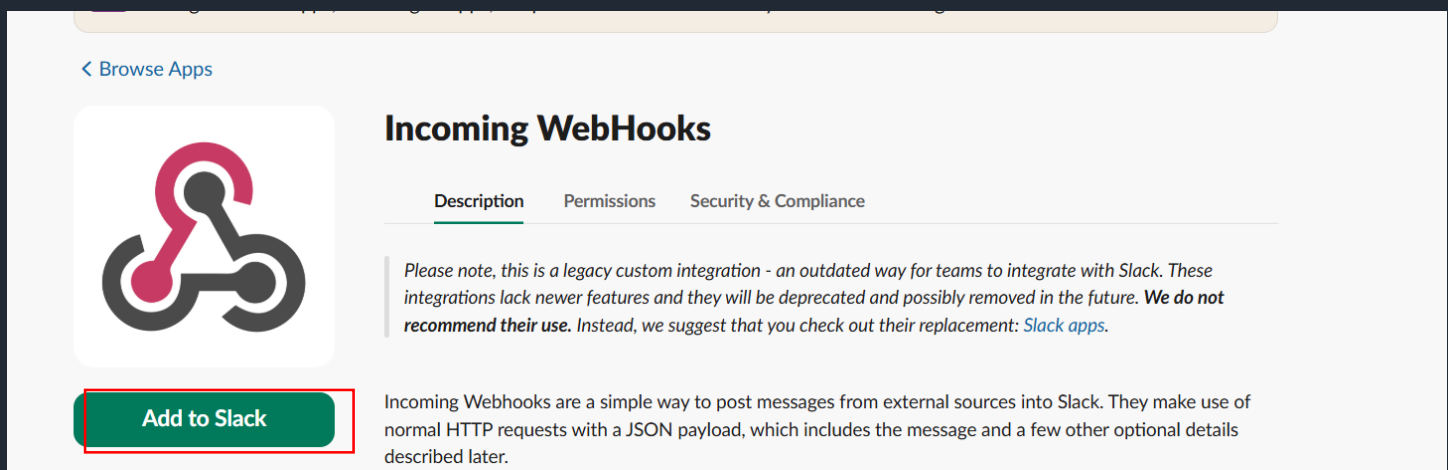
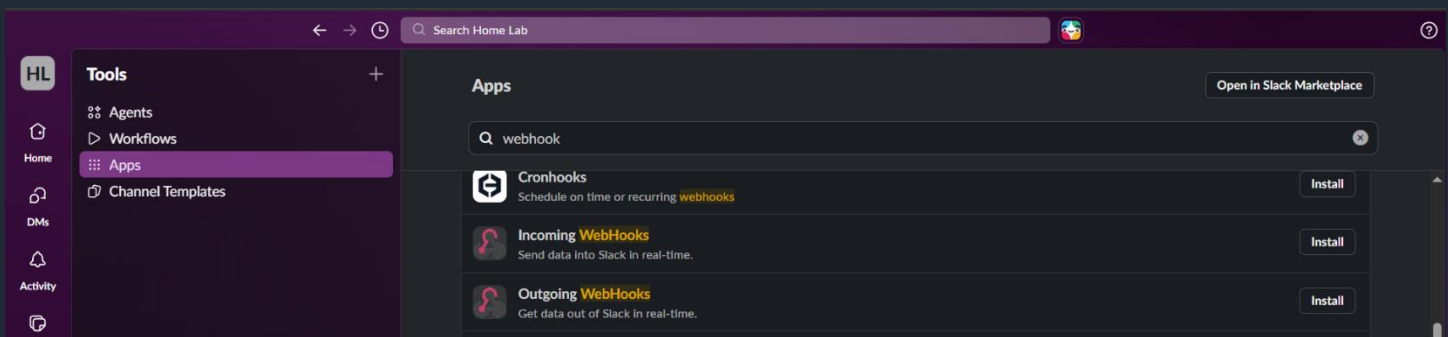
Sending Alerts

Alerting rules allow you to define alert conditions based on Prometheus expression language expressions and to send notifications about firing alerts to an external service. Whenever the alert expression results in one or more vector elements at a given point in time, the alert counts as active for these elements' label sets.

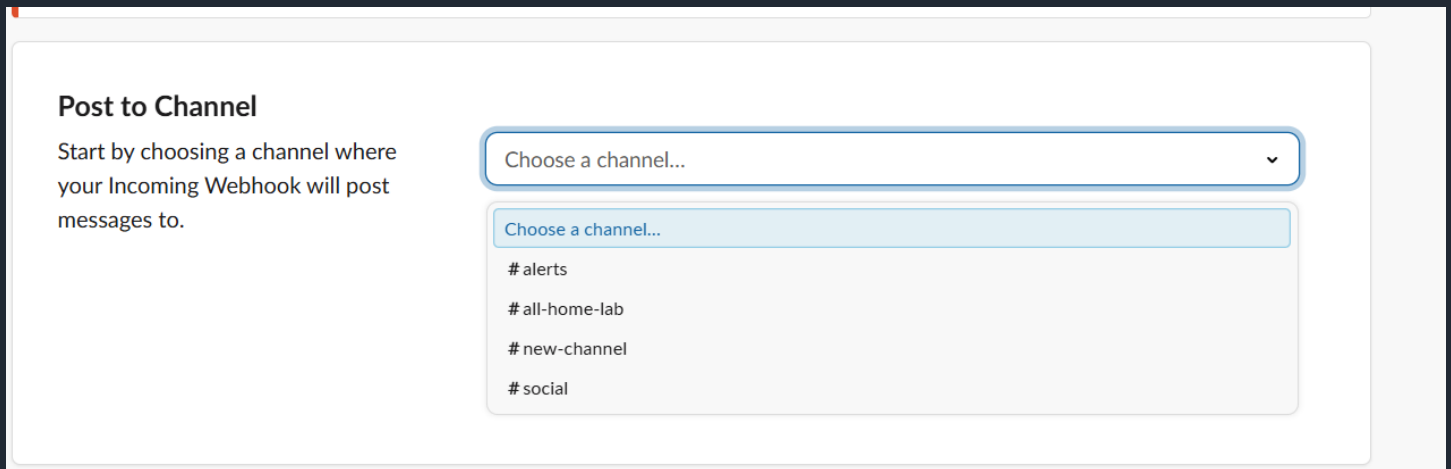
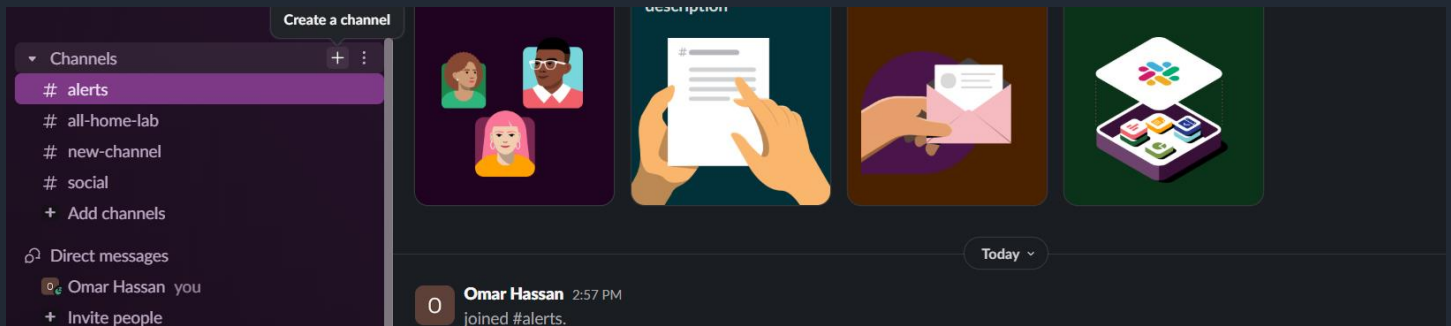
Alerting with Prometheus is separated into two parts. Alerting rules in Prometheus servers send alerts to an Alertmanager. The Alertmanager then manages those alerts, including silencing, inhibition, aggregation and sending out notifications via methods such as email, on-call notification systems, and chat platforms.

First let's go to slack and create a workspace.

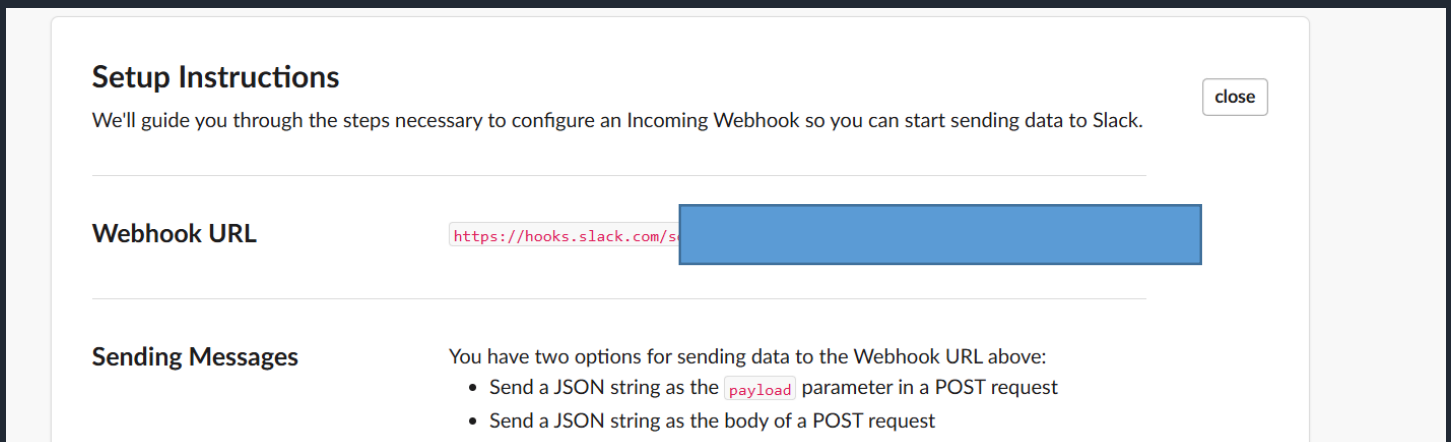
Then Apps > Incoming web hooks



Make sure to create a channel to receive the alerts on



What we need right now is the webhook URL



Then let's get to configuring the alert-manager.

```
$ sudo mkdir -p /var/snap/docker/common/alertmanager
$ sudo tee /var/snap/docker/common/alertmanager/alertmanager.yml << 'EOF'
global:
  slack_api_url: 'YOUR WEBHOOK URL'
```



```
title: '{{ .CommonAnnotations.summary }}'
text: '{{ .CommonAnnotations.description }}'
send_resolved: true
```

Now onto creating the rules for Prometheus

I am going to create an alert for brute force (just for testing) and for commands executed and an extra in case the container fails.

```
$ sudo tee /var/snap/docker/common/prometheus-data/alert_rules.yml <<
'EOF' groups: - name: cowrie_alerts rules: - alert: BruteForceDetected
expr: increase(cowrie_login_attempts_total{result="fail"}[1m]) > 3 for: 1m
labels: severity: warning annotations: summary: "Brute force attack
detected" description: "More than 3 failed login attempts in 1 minute -
username: '{{ $labels.username }}' - alert: AttackerRunningCommands expr:
increase(cowrie_command_total[5m]) > 0 for: 0m labels: severity: critical
annotations: summary: "Attacker running commands in honeypot" description:
"Command '{{ $labels.command }}' executed on sensor '{{ $labels.sensor }}'"
- alert: ContainerDown expr: absent(up{job="scrapers"}) or
up{job="scrapers"} == 0 for: 1m labels: severity: critical annotations:
summary: "Container is down" description: "A monitored container has
stopped responding on '{{ $labels.instance }}'" EOF
```

Now updating Prometheus config to load rules and point to the alert-manager.

```
$ sudo tee /var/snap/docker/common/prometheus-data/prometheus.yaml <<
'EOF'
```

global:

```
  scrape_interval: 5s
  evaluation_interval: 5s
```

alerting:

```
  alertmanagers:
    - static_configs:
      - targets: ['alertmanager:9093']
```

rule_files:

```
  - /etc/prometheus/alert_rules.yml
```

```
scrape_configs:
  - job_name: 'cowrie'
    static_configs:
      - targets: ['cowrie:9000']

  - job_name: 'scrapers'
    static_configs:
      - targets:
        - 'node-exporter:9100'
        - 'cadvisor:8080'
```

EOF

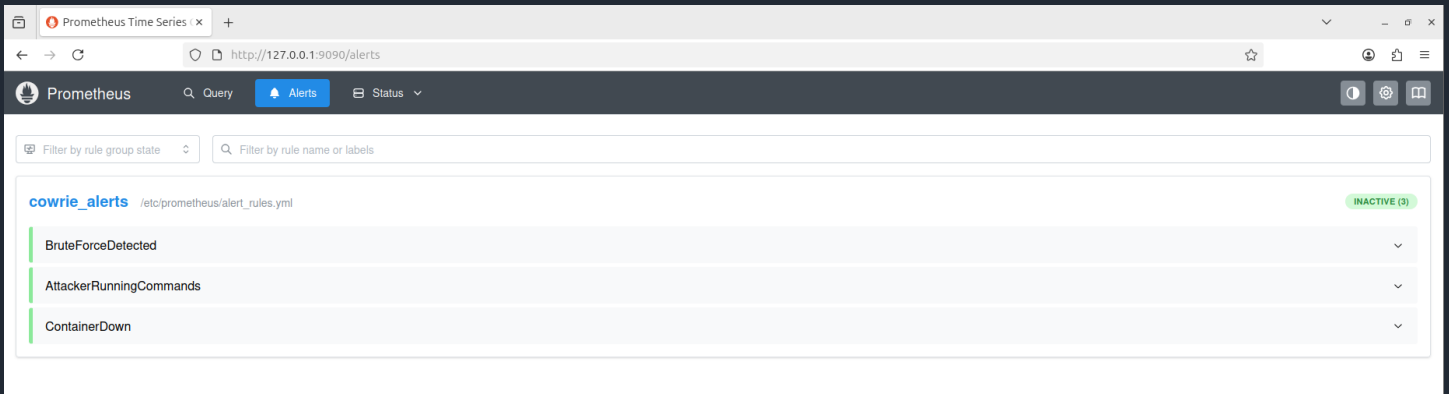
Run alert-manager container

```
$ docker run -d \
  --name alertmanager \
  --network cowrie-net \
  --restart unless-stopped \
  -p 9093:9093 \
  -v
/var/snap/docker/common/alertmanager/alertmanager.yml:/alertmanager.yml \
  prom/alertmanager \
  --config.file=/alertmanager.yml
```

Checking logs

```
$ sudo docker logs alertmanager
```

```
root@honeypot-VMware-Virtual-Platform:/home/honeypot# docker logs alertmanager
time=2026-05-03T14:32:55.408Z level=INFO source=main.go:193 msg="Starting Alertmanager" version="(version=0.32.1, branch=HEAD, revision=8768aa6f65f1a888b5aa5fbf877cf20ad45d1f61)"
time=2026-05-03T14:32:55.409Z level=INFO source=main.go:196 msg="Build context" build_context="(go=go1.26.2, platform=linux/amd64, user=root@6f5b35dc1248, date=20260429-17:34:54, tags=netgo)"
time=2026-05-03T14:32:55.410Z level=INFO source=cluster.go:192 msg="setting advertise address explicitly" component=cluster addr=172.18.0.6 port=9094
time=2026-05-03T14:32:55.412Z level=INFO source=cluster.go:682 msg="Waiting for gossip to settle..." component=cluster interval=2s
time=2026-05-03T14:32:55.449Z level=INFO source=coordinator.go:111 msg="Loading configuration file" component=configuration file=/alertmanager.yml
time=2026-05-03T14:32:55.451Z level=INFO source=coordinator.go:124 msg="Completed loading of configuration file" component=configuration file=/alertmanager.yml
time=2026-05-03T14:32:55.456Z level=INFO source=tls_config.go:354 msg="Listening on" address=[:]:9093
time=2026-05-03T14:32:55.456Z level=INFO source=tls_config.go:357 msg="TLS is disabled." http2=false address=[:]:9093
time=2026-05-03T14:32:57.413Z level=INFO source=cluster.go:707 msg="gossip not settled" component=cluster polls=0 before=0 now=1 elapsed=2.000987775s
time=2026-05-03T14:33:05.416Z level=INFO source=cluster.go:699 msg="gossip settled; proceeding" component=cluster elapsed=10.004633692s
```



Now let's use our kali machine to brute force our way into the honeypot.

```
[kali@kali]~$  
_ $ for i in {1..10}; do  
for> ssh -p 22 -o StrictHostKeyChecking=no db_admin@192.168.25.135 2>/dev/null  
for> done  
db_admin@192.168.25.135's password:  
db_admin@192.168.25.135's password:  
db_admin@192.168.25.135's password:  
db_admin@192.168.25.135's password:  
db_admin@192.168.25.135's password:  
db_admin@192.168.25.135's password:  
db_admin@192.168.25.135's password:  
db_admin@192.168.25.135's password:  
db_admin@192.168.25.135's password:  
db_admin@192.168.25.135's password:  
db_admin@192.168.25.135's password:  
  
ATTENTION: THIS SYSTEM IS MONITORED.  
  
Thank you for logging in. Please remember that all actions are recorded  
for security purposes. Report any suspicious behavior to the administrator.  
db_admin@svr-web01:~$
```

Filter by rule group state		Filter by rule name or labels	
cowrie_alerts /etc/prometheus/alert_rules.yml		FIRING (1) PENDING (1) INACTIVE (1)	
BruteForceDetected		PENDING (1) ▾	
AttackerRunningCommands		FIRING (1) ▾	
ContainerDown		▾	

As you can see the brute force alert was triggered as well as the running commands alert as I ran a few commands after logging in.

Now let's go to Slack.



Alertmanager APP 6:22 PM

Attacker running commands in honeypot

Command 'ls' executed on sensor svr-web01

We can customize this more but I just wanted to stick with a simple demo for the lab.

And that's it for this lab I will continue working on it adding extra stuff like forensics artifacts acquisition and automation using n8n and LLMs but that will be in another documentation.